

SkillPulse: 9-Phase Implementation Plan

Job Market Signal Pipeline — Reference Roadmap

Phase 1 — Foundation & Ingestion

Goal: Reliably collect raw job-posting data from external sources

1. Create the Git repository and professional folder structure.
2. Set up Python environment, dependency management, .env, .gitignore, and configuration files.
3. Define source contracts: what each API returns, required fields, limits, pagination behavior, and authentication method.
4. Create API credentials for Adzuna and validate RemoteOK access.
5. Build reusable HTTP client logic with timeout, retry, exponential backoff, and error handling.
6. Build one ingestion module per source.
7. Implement pagination and configurable search parameters such as country, role, category, and page size.
8. Save each API response unchanged in a raw landing zone, with source name and ingestion timestamp.
9. Add idempotency controls so the same run does not create uncontrolled duplicate raw files.
10. Add structured logs for each request, page, failure, and record count.
11. Containerize the ingestion service.
12. Add basic unit tests using mocked API responses.

Main deliverables: Working API ingestion scripts; raw JSON landing zone; configuration system; Dockerized ingestion service; ingestion logs; basic tests.

Phase 2 — Orchestration

Goal: Schedule and control ingestion jobs automatically

1. Add Apache Airflow to Docker Compose.
2. Configure Airflow metadata database and local executor setup.
3. Create a DAG for each source or one DAG with source-specific tasks.
4. Define task dependencies: validate config → ingest source → validate raw output → report run status.
5. Add schedules, initially daily or every 12 hours.
6. Configure retries, retry delays, timeouts, and task-level failure behavior.
7. Pass run date and source parameters safely into tasks.
8. Add failure notifications through logs first, then email or another free alert method.
9. Add backfill support for historical runs.
10. Test manual trigger, scheduled trigger, retry behavior, and failed-task recovery.

Main deliverables: Airflow stack; scheduled DAGs; retry and failure logic; run history; backfill process.

Phase 3 — Data Modeling & Storage

Goal: Convert raw API data into a queryable warehouse

1. Add PostgreSQL as the main warehouse database.
2. Define three layers: raw, staging, and marts.
3. Load raw JSON metadata and payload references into raw tables.
4. Create a dbt project connected to PostgreSQL.
5. Build staging models that flatten API JSON into clean columns.

6. Standardize dates, company names, location fields, URLs, salary fields, and source identifiers.
7. Design a dimensional model: fact_job_postings, dim_date, dim_company, dim_location, dim_source, and later dim_skill.
8. Define stable business keys and surrogate keys.
9. Add incremental models so only new or changed data is processed.
10. Add database indexes for frequent filters and joins.
11. Document each model and column in dbt YAML.
12. Validate row counts from raw to staging to warehouse.

Main deliverables: PostgreSQL warehouse; dbt project; raw/staging/mart schemas; dimensional model; incremental transformations.

Phase 4 — Skill & Entity Extraction

Goal: Turn job text into structured labor-market signals

1. Create a versioned skills taxonomy in CSV or YAML.
2. Include canonical skill names, aliases, categories, and tool families.
3. Start with deterministic extraction using normalized text, keyword matching, regex, and phrase boundaries.
4. Prevent false matches using exclusion rules and word boundaries.
5. Extract seniority level from titles and descriptions.
6. Extract work mode: remote, hybrid, onsite, or unknown.
7. Parse salary currency, minimum, maximum, period, and normalized annual estimate when possible.
8. Parse location into country, city, region, and remote scope where available.
9. Create a bridge table between job postings and skills because one posting can contain many skills.
10. Store extraction version, confidence, matched phrase, and processing timestamp for auditability.
11. Add test cases for known job descriptions.
12. Measure extraction quality manually on a sample before adding NLP.

Main deliverables: Skills taxonomy; extraction pipeline; posting-skill bridge table; structured seniority, salary, location, and work-mode fields; extraction tests.

Phase 5 — Data Quality & Testing

Goal: Detect bad data before it reaches analytics

1. Define data-quality rules by layer: raw, staging, and marts.
2. Add dbt tests for uniqueness, non-null values, relationships, accepted values, and freshness.
3. Add custom SQL tests for duplicate postings, impossible salaries, invalid dates, broken URLs, and missing source IDs.
4. Add volume checks to detect unexpected drops or spikes in records.
5. Add schema-drift detection for API fields that appear, disappear, or change type.
6. Create a quarantine strategy for malformed records instead of silently deleting them.
7. Add data-quality summary tables or logs per pipeline run.
8. Use Great Expectations only where it provides clear value, rather than adding it everywhere.
9. Define severity levels: warning, failed check, and blocking failure.
10. Test deliberate bad inputs to confirm failures are visible.

Main deliverables: dbt test suite; custom SQL checks; quality reports; schema-drift handling; quarantine process.

Phase 6 — Trend Analytics Layer

Goal: Produce reliable metrics about skill demand over time

1. Define metric rules before writing SQL. Example: skill demand means count of distinct active job postings mentioning a skill.
2. Build daily and monthly demand aggregates by skill.
3. Build skill demand by role, location, company, source, and work mode.
4. Build growth metrics using window functions: month-over-month, quarter-over-quarter, and rolling averages.
5. Add minimum-volume thresholds so small samples do not produce misleading growth percentages.
6. Build salary analytics by skill, seniority, location, and work mode.
7. Build co-occurrence analysis: skills frequently mentioned together.
8. Build data freshness and source coverage metrics.
9. Materialize high-use tables for dashboard performance.
10. Document metric definitions and limitations.
11. Validate a sample of metrics manually against source postings.

Main deliverables: dbt analytics marts; growth metrics; demand rankings; salary analysis; skill co-occurrence metrics; metric dictionary.

Phase 7 — Serving Layer

Goal: Make the intelligence usable through a dashboard and optional API

1. Select Streamlit for a code-first portfolio dashboard, or Metabase for faster BI exploration.
2. Connect only to curated analytics marts, not raw tables.
3. Build dashboard filters for date range, skill, location, role, source, and work mode.
4. Build core views: headline KPIs, skill rankings, trend lines, regional demand, salary patterns, and skill co-occurrence.
5. Add clear definitions beside metrics to prevent misinterpretation.
6. Add empty-state and low-data handling.
7. Optimize queries through pre-aggregations and indexes.
8. Optionally add FastAPI endpoints for read-only metrics.
9. Add API response schemas, pagination, validation, and documentation if FastAPI is used.
10. Test dashboard behavior with realistic data volume.

Main deliverables: Dashboard; interactive filters; trend and ranking views; optional FastAPI service; user-facing metric definitions.

Phase 8 — Deployment & Monitoring

Goal: Run the full system unattended and observe failures

1. Consolidate all services in Docker Compose: ingestion, Airflow, PostgreSQL, dbt execution, dashboard, and optional API.
2. Separate local, development, and production-like configuration.
3. Move secrets into environment variables or a secure local secret method; never commit them.
4. Add structured JSON logging where useful.
5. Add health checks for database, Airflow, and dashboard services.
6. Monitor pipeline run success, duration, row counts, freshness, and quality-test results.
7. Configure practical alerts for failed DAGs and repeated failures.
8. Add database backup and retention procedures.
9. Add raw-data retention policy to control storage growth.
10. Add basic CI checks for formatting, tests, and dbt compilation.

11. Write an operational runbook for common failures.

Main deliverables: Production-like Docker Compose stack; health checks; logs; alerts; backup policy; CI basics; operational runbook.

Phase 9 — Documentation & Case Study

Goal: Turn the build into a credible engineering portfolio project

1. Write a clear README with project problem, architecture, setup, commands, and screenshots.
2. Create an architecture diagram showing sources, ingestion, orchestration, storage layers, transformation, analytics, and dashboard.
3. Create a data-flow diagram showing raw-to-mart movement.
4. Write a data dictionary for important tables and fields.
5. Document API assumptions, rate limits, data gaps, and legal or terms-of-use constraints.
6. Document major engineering decisions and tradeoffs.
7. Add testing and monitoring evidence.
8. Add a short demo walkthrough with example business questions.
9. Write a portfolio case study focused on impact, architecture, reliability, and lessons learned.
10. Create a future roadmap: more sources, stronger extraction, streaming ingestion, cloud deployment, and ML-based skill normalization.

Main deliverables: README; diagrams; data dictionary; runbook; screenshots; portfolio case study; future roadmap.